折腾笔记[17]-使用rust创建linux系统服务

摘要

使用rust内嵌配置文件, 创建新用户并创建linux的service服务;实现后台服务循环打印时间到 `/tmp/log_file_service`.
Use Rust to embed configuration files, create new users, and create service services for Systemd; Implement a backend service loop to print time to `/tmp/log_file_Service`

关键词

rust;linux;systemctl;embed_file;zigbuild;

关键信息

- ✨项目地址:[https://github.com/ByeIO/byeefree.esp32s3_p2p.embassy/tree/rust/rust_linux2linux_udp]

```
[package]
name = "rust_linux2linux_udp"
version = "0.1.0"
edition = "2021"
authors = ["qsbye"]

[dependencies]
# base64编码
base64 = "0.22.1"
# 命令行cli工具
clap = { version = "4.5.32", features = ["derive"] }
# 网络协议栈
edge-net = { version = "0.9.3", features = ["std"] }
# systemd操作接口
# libsystemd = { version = "0.7.0", path = "./static/libsystemd" }
systemctl = "0.4.0"
# 异步框架
tokio = "1.44.1"
# usb设备通信(纯rust)
nusb = { version = "0.2.0-beta", path = "./static/nusb" }
# 虚拟网卡接口
tappers = "0.4.2"
# 内嵌文件
embed-file = "0.2.0"
# 日志记录
env_logger = "0.11.7"
```

原理简介

cargo-zigbuild简介

[https://github.com/rust-cross/cargo-zigbuild]
Compile Cargo project with zig as linker for easier cross compiling.

```
cargo install --locked cargo-zigbuild
pip install cargo-zigbuild
```

embed-file简介

[https://crates.io/crates/embed-file]
注意, 需要--release参数编译才会嵌入文件
A simplified version of rust-embed for single files instead of folders. Embeds files content into the binary in release mode, but loads it from the fs in debug.

The goal of is to avoid unnecessary recompilations when an included by e.g. [include_str] file change.

```
let my_string = embed_file::embed_string!("path/to/my-text.txt");
let my_image = embed_file::embed_bytes!("path/to/my-image.jpg");
```

linux的service/daemon简介

[https://blog.kelu.org/tech/2023/12/11/mac-systemd-like-launchd.html]
[https://docs.rs/systemctl/latest/systemctl/]
[https://www.reddit.com/r/linuxquestions/comments/z9mk8f/whats_the_difference_between_a_daemon_and_a/]
[https://www.cnblogs.com/kljh/p/16743836.html]
你提到的关于“Daemon”和“Service”的讨论确实很有意思。这两个术语确实在操作系统领域有着深厚的历史背景，尽管它们起源于不同的系统（Unix和Windows），但它们的核心概念是相似的：都是指在后台运行的程序，通常不与用户直接交互，且往往在更高的权限下运行。

关于它们是否“一直在做某事”或“等待做某事”的争论，确实有些过于简化了。实际上，无论是Daemon还是Service，它们的设计目的都是为了在后台持续运行，处理系统任务或响应特定事件。至于那些按计划或触发条件运行一次就退出的程序，确实更倾向于被称为“任务”或“作业”，而不是“服务”或“守护进程”。

在Windows中，Task Scheduler是一个强大的工具，用于安排和管理这些一次性任务，而Service Manager则更适合管理那些需要持续运行的服务。类似地，在Linux中，cron和systemd等工具也分别用于处理定时任务和守护进程。

总的来说，术语的使用确实存在一定的模糊性，但理解它们的核心功能和设计目的才是关键。希望这些信息能帮助你更好地理解这些概念！

They are the same thing - and disinformation and misunderstanding surrounds both
I see that some answers are getting fancy with explanations involving systemd.

Both words predate systemd by decades.

Or whether they are "always doing something" or "waiting to do something" - the dumbest thing I've heard so far today.

"Daemon" originated in the unix world, "Service" in Windows. Both refer to programs that run and stay running, usually in a way that the user doesn't interact with (eg in a different user context, often more privileged, eg root or SYSTEM).

If it's designed just run occasionally or on a schedule or trigger and then quit, then yes terminology gets fuzzy and gray. Someone said that's still a service/daemon. Maybe - but many IT pros, possibly most, would disagree - even though such programs may in fact be launched by Windows "Service Manager" or systemd. (The former is often grossly misused/abused. The latter has never been limited to dealing with just daemons.) But sure, there's room for debate, and agreeing on a hard distinction isn't going to make or break society.

There are countless ways to launch tasks at the system level that do something and then quit - eg Windows Task Scheduler, which is complex and heavily used. Countless things stuffed into Windows Service Manager should instead go in there (possibly because Task Scheduler is honestly complex, fiddly, and defined tasks are prone to breaking with system changes). Or with Linux, you have crontab, cron.daily (etc.), etc. Nobody - at least nobody that matters or that I'm aware of - considers those "services" or "daemons". More like "Tasks", "Jobs", "Cronjobs", etc.

In my view, a daemon is just a kind of process that forks and keeps running "in the background" (as a child process of the process which exec'd it) while the "foreground" process exits immediately allowing further execution of the parent program while the child keeps running "in the background".

A service encapsulates a process into a higher abstraction unit that has metadata, surrounding semantics (i.e. what should be done when it fails, what should be done before it starts and a whole lot more) and can be managed by a service manager like systemd.

A daemon can be part of a service and historically was but a service does not need to include a daemon nowadays.
A service's process(es) can keep running without technically forking into the background because systemd can deal with that. It's actually better to not daemonise because, if there are further processes spawned, it's clearer which one is the "main" process of a service. Dunno about other service managers.

macOS的服务配置文件plist简介

[https://blog.kelu.org/tech/2023/12/11/mac-systemd-like-launchd.html]
在 macOS 中，Launchd 是一个系统级别的进程管理工具，用于启动、停止和管理系统和用户级别的进程。Launchd 进程本身是由内核启动的，并负责启动其他进程，包括系统服务和用户级别的进程。macOS 中有两种类型的守护进程，一种是系统级别的（Daemons），一种是用户级别的（Agents），它们的配置文件放的位置是不一样的。

frpc.plist

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE plist PUBLIC "-//Apple//DTD PLIST 1.0//EN" "http://www.apple.com/DTDs/PropertyList-
<plist version="1.0">
<dict>
  <key>KeepAlive</key>
  <true/>
  <key>Label</key>
  <string>frpc</string>
  <key>ProgramArguments</key>
  <array>
    <string>/Users/kelu/Workspace/bin/frp_0.52.3_darwin_arm64/frpc</string>
    <string>-c</string>
    <string>/Users/kelu/Workspace/bin/frp_0.52.3_darwin_arm64/tmp.toml</string>
  </array>
  <key>RunAtLoad</key>
  <true/>
</dict>
</plist>
```

rust代码中执行系统命令

[https://doc.rust-lang.org/std/process/struct.Command.html]
A process builder, providing fine-grained control over how a new process should be spawned.

A default configuration can be generated using Command::new(program), where program gives a path to the program to be executed. Additional builder methods allow the configuration to be changed (for example, by adding arguments) prior to spawning:

```
use std::process::Command;

let output = if cfg!(target_os = "windows") {
    Command::new("cmd")
        .args(["/C", "echo hello"])
        .output()
        .expect("failed to execute process")
} else {
    Command::new("sh")
        .arg("-c")
        .arg("echo hello")
        .output()
        .expect("failed to execute process")
};

let hello = output.stdout;
```

实现

1. 后台服务: log_time
2. 服务安装器: service_installer

- 总体流程: service_installer将内嵌的文件放置到对应位置, 然后配置systemctl启动服务并设置开机启动.

交叉编译(使用cargo-zigbuild替代musl解决libc问题):

```
cargo zigbuild --release --example log_file_service --target aarch64-unknown-linux-gnu
cargo zigbuild --release --example service_installer --target aarch64-unknown-linux-gnu
```

log_time.rs

```
// 导入需要的标准库模块
use std::{
    fs::OpenOptions,
    io::Write,
    thread,
    time::{SystemTime, UNIX_EPOCH}
};

fn main() {
    // 循环写入时间到文件
    loop {
        // 获取当前UNIX时间戳（秒）
        let timestamp = SystemTime::now()
            .duration_since(UNIX_EPOCH)
            .expect("获取时间失败")
            .as_secs();

        // 以覆写模式打开文件（不存在则创建，存在则清空）
        let mut file = OpenOptions::new()
            .create(true)
            .write(true)
            .truncate(true)
            .open("/tmp/log_file_service")
            .expect("无法打开文件");

        // 写入时间戳并换行
        writeln!(file, "{}", timestamp)
            .expect("写入文件失败");

        // 每秒执行一次
        thread::sleep(std::time::Duration::from_secs(1));
    }
}
```

service_installer.rs

RS 折叠 复制 全屏

```
#![allow(unused)]

//! byeefree_log_time服务安装器

// 标准库
use std::{fs, path::Path};
use std::error::Error;
use std::process::{Command, Stdio};
use std::io;
use std::os::unix::fs::PermissionsExt;
use std::env::consts::OS;

// 嵌入文件
use embed_file::embed_bytes;

fn main()-> Result<(), Box<dyn Error>>{
    // 新建用户组_ByeIO_和用户_Byeefree_
    let os_name = std::env::consts::OS;

    println!("当前操作系统为:{}", os_name);

    match os_name {
        "linux" => linux_create_user_group()?,
        "macos" => macos_create_user_group()?,
        _ => return Err(Box::new(io::Error::new(
            io::ErrorKind::Unsupported,
            format!("Unsupported OS: {}", os_name)
        )))
    };

    // 复制可执行文件和.service配置文件到对应目录
    /// 嵌入二进制文件到编译产物中
    let service_file = embed_bytes!("../assets/byeefree_log_time.service");
    let plist_file = embed_bytes!("../assets/byeefree_log_time.plist");
    let exec_file = embed_bytes!("../assets/byeefree_log_time_service.exec");
    let elf_file = embed_bytes!("../assets/byeefree_log_time_service.linux.aarch64");

    /// 复制文件
    println!("复制文件中...");
    // 创建目标目录（如果不存在）
    let output_dir_1 = Path::new("/tmp/_Byeefree_");
    let output_dir_2 = Path::new("/etc/systemd/system");
    let output_dir_3 = Path::new("/Library/LaunchDaemons");
    let output_dir_4 = Path::new("/usr/local/bin");

    // 递归创建目录
    fs::create_dir_all(output_dir_1)?;

    // 构建输出文件路径
    let output_service_path = output_dir_2.join("byeefree_log_time.service");
    let output_plist_path = output_dir_3.join("byeefree_log_time.plist");
    let output_exec_path = match os_name{
        "linux" => { output_dir_4.join("byeefree_log_time_service.exec") },
        "macos" => { output_dir_1.join("byeefree_log_time_service.exec") },
        _ => { return Err("unknown OS".into()); },
    };

    // 将嵌入的二进制数据写入文件
    match os_name {
        "linux" => {
            // 递归创建目录
            fs::create_dir_all(output_dir_2)?;
            fs::write(&output_service_path, service_file)?;
            println!("文件已保存至: {}", output_service_path.display());
            fs::write(&output_exec_path, elf_file)?;
            println!("文件已保存至: {}", output_exec_path.display());
        },
        "macos" => {
            // 递归创建目录
            fs::create_dir_all(output_dir_3)?;
            fs::write(&output_plist_path, plist_file)?;
            println!("文件已保存至: {}", output_plist_path.display());
            fs::write(&output_exec_path, exec_file)?;
            println!("文件已保存至: {}", output_exec_path.display());
        },
        _ => {},
    }

    // 配置可执行文件权限及所有权
    fs::set_permissions(output_exec_path.clone(), fs::Permissions::from_mode(0o755))?;

    let chown_status = Command::new("chown")
        .arg("_Byeefree_:_ByeIO_")
        .arg(output_exec_path)
        .status()?;
    if !chown_status.success() {
        return Err("Failed to set executable ownership".into());
    }else{
        println!("文件添加可执行权限成功.");
    }

    // 设置服务文件权限（仅限Linux）
    match os_name {
        "linux" => {
            fs::set_permissions(output_service_path.clone(), fs::Permissions::from_mode(0o644
        },
        "macos" => {
            fs::set_permissions(output_plist_path.clone(), fs::Permissions::from_mode(0o644))
        },
        _ => {},
    }

    println!("服务配置文件添加权限成功.");

    // 启动服务并设置开机自启
    match os_name {
        "linux" => {
            Command::new("systemctl")
                .args(&["daemon-reload"])
                .status()?;
            println!("linux服务配置重载成功.");
            Command::new("systemctl")
                .args(&["start", "byeefree_log_time.service"])
                .status()?;
            println!("linux服务启动成功.");
            Command::new("systemctl")
                .args(&["enable", "byeefree_log_time.service"])
                .status()?;
            println!("linux服务设置开机自启成功.");
        },
        "macos" => {
            Command::new("launchctl")
                .args(&["load", "-w", output_plist_path.clone().to_str().unwrap()])
                .status()?;
            println!("macOS服务设置开机自启成功.");
        },
        _ => {}
    }

    // 全部任务完成
    println!("安装服务完成, 输入`cat /tmp/log_file_service`查看效果");
    Ok(())
}

fn linux_create_user_group() -> io::Result<()> {
    use std::io::{self, ErrorKind, Error};

    // 检查用户组是否存在
    let group_exists = Command::new("getent")
        .arg("group")
        .arg("_ByeIO_")
        .status()?
        .success();

    if !group_exists {
        // 创建用户组
        let group_output = Command::new("groupadd")
            .arg("_ByeIO_")
            .output()?;

        if !group_output.status.success() {
            return Err(Error::new(
                ErrorKind::Other,
                format!("用户组创建失败: {}", String::from_utf8_lossy(&group_output.stderr))
            ));
        }
    }

    // 检查用户是否存在
    let user_exists = Command::new("id")
        .arg("-u")
        .arg("_Byeefree_")
        .status()?
        .success();

    if !user_exists {
        // 创建用户
        let user_output = Command::new("useradd")
            .args(&["-g", "_ByeIO_", "_Byeefree_"])
            .output()?;

        if !user_output.status.success() {
            return Err(Error::new(
                ErrorKind::Other,
                format!("用户创建失败: {}", String::from_utf8_lossy(&user_output.stderr))
            ));
        }
    }

    Ok(())
}

fn macos_create_user_group() -> io::Result<()> {
    use std::io::{self, ErrorKind, Error};

    // 检查用户组是否存在
    let group_exists = Command::new("sudo")
        .args(&["dscl", ".", "-read", "/Groups/_ByeIO_"])
        .status()
        .map(|s| s.success())?;

    if !group_exists {
        // 创建用户组
        let group_create = Command::new("sudo")
            .args(&["dscl", ".", "-create", "/Groups/_ByeIO_"])
            .status()?;

        if !group_create.success() {
            return Err(Error::new(ErrorKind::Other, "macOS用户组创建失败"));
        }
    }

    // 检查用户是否存在
    let user_exists = Command::new("sudo")
        .args(&["dscl", ".", "-read", "/Users/_Byeefree_"])
        .status()
        .map(|s| s.success())?;

    if !user_exists {
        // 创建用户属性
        let user_commands = [
            ("/Users/_Byeefree_", "UserShell", "/bin/bash"),
            ("/Users/_Byeefree_", "RealName", "ByeIO Service Account"),
            ("/Users/_Byeefree_", "PrimaryGroupID", "2000"),
        ];

        for (path, key, value) in &user_commands {
            let status = Command::new("sudo")
                .args(&["dscl", ".", "-create", path, key, value])
                .status()?;
            if !status.success() {
                return Err(Error::new(
                    ErrorKind::Other,
                    format!("用户属性创建失败: {}/{}/{}", path, key, value)
                ));
            }
        }

        // 将用户加入组
        let append_status = Command::new("sudo")
            .args(&["dscl", ".", "-append", "/Groups/_ByeIO_", "GroupMembership", "_Byeefree_
            .status()?;
        if !append_status.success() {
            return Err(Error::new(ErrorKind::Other, "无法将用户添加到组"));
        }
    }

    Ok(())
}
```

服务文件:
log_time.service

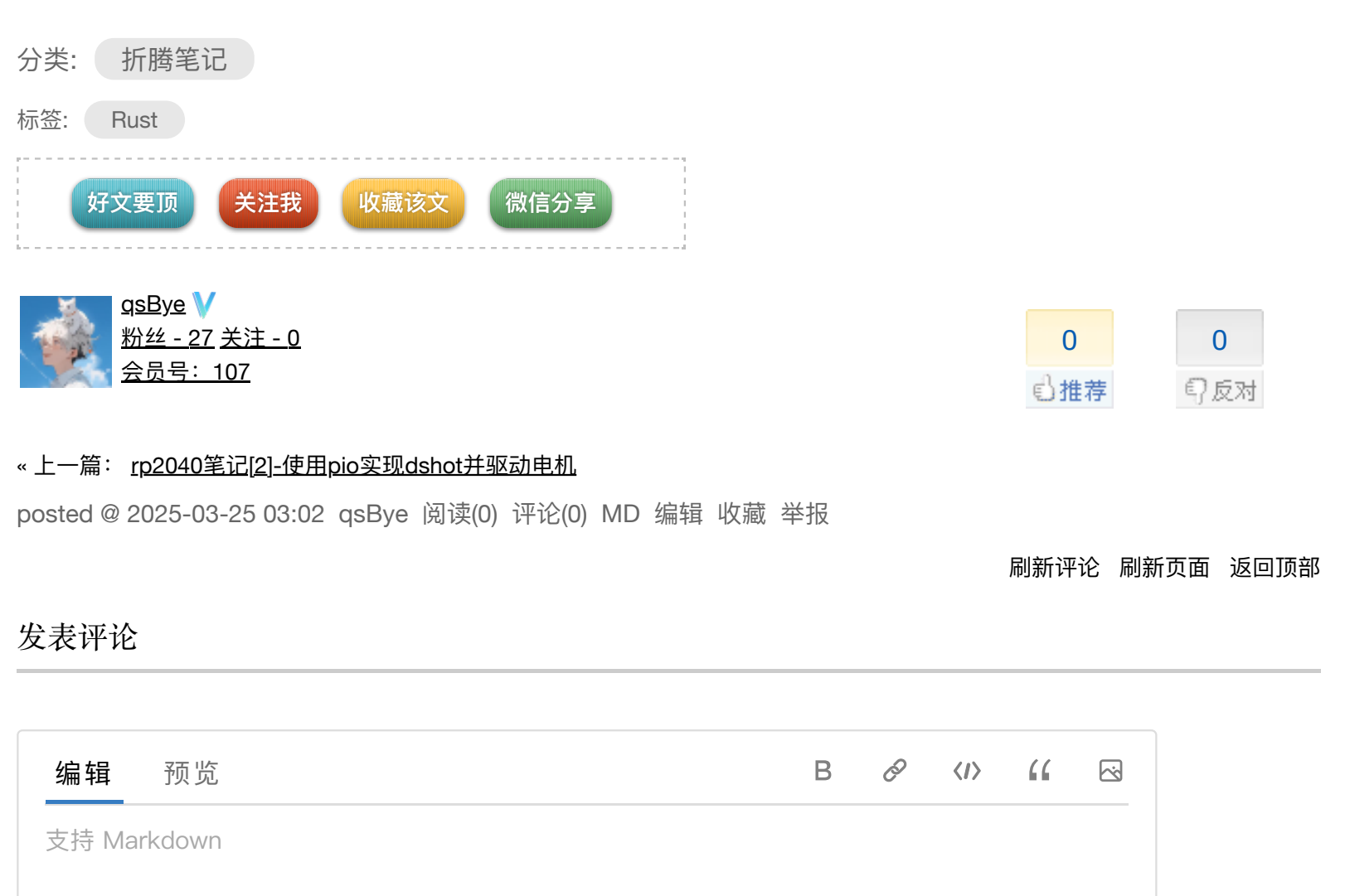
```
[Unit]
Description=ByeIO Byeefree Log Time Service
After=network.target

[Service]
# 可指定运行用户（建议使用非root用户）
User=_Byeefree_
# 用户组
UserGroup=_ByeIO_
# 工作目录设为/tmp
WorkingDirectory=/tmp
# 可执行文件路径
ExecStart=/usr/local/bin/byeefree_log_time_service.exec
# 异常退出时自动重启
Restart=always
# 重启间隔
RestartSec=3

[Install]
WantedBy=multi-user.target
```

效果

服务配置成功:

```
office:~:% sudo ./service_installer
_ByeIO_:x:1001:
1001
复制文件中...
文件已保存至: /etc/systemd/system/byeefree_log_time.service
文件已保存至: /tmp/_Byeefree_/byeefree_log_time_service.exec
文件添加可执行权限成功.
服务配置文件添加权限成功.
linux服务配置重载成功.
linux服务启动成功.
Created symlink /etc/systemd/system/multi-user.target.wants/byeefree_log_time.service → /etc/
linux服务设置开机自启成功.
安装服务完成, 输入`cat /tmp/log_file_service`查看效果
```

服务启动成功



```
[office:~:% cat /tmp/log_file_service
1742841566
[office:~:% cat /tmp/log_file_service
1742841572
[office:~:% cat /tmp/log_file_service
1742841573
[office:~:% cat /tmp/log_file_service
1742841574
[office:~:% cat /tmp/log_file_service
1742841575
[office:~:% sudo systemctl status byeefree_log_time.service
● byeefree_log_time.service - ByeIO Byeefree Log Time Service
     Loaded: loaded (/etc/systemd/system/byeefree_log_time.service; enable>
     Active: active (running) since Tue 2025-03-25 02:39:10 HKT; 32s ago
   Main PID: 28200 (byeefree_log_ti)
      Tasks: 1 (limit: 4526)
     Memory: 192.0K
        CPU: 19ms
     CGroup: /system.slice/byeefree_log_time.service
             └─28200 /usr/local/bin/byeefree_log_time_service.exec

Mar 25 02:39:10 office systemd[1]: Started ByeIO Byeefree Log Time Service.
lines 1-11/11 (END)
```

分类: 折腾笔记
标签: Rust

好文要顶 关注我 收藏该文 微信分享

qsBye
粉丝 - 27 关注 - 0
会员号：107

0 推荐 0 反对

« 上一篇： rp2040笔记[2]-使用pio实现dshot并驱动电机

posted @ 2025-03-25 03:02 qsBye 阅读(0) 评论(0) MD 编辑 收藏 举报

刷新评论 刷新页面 返回顶部

发表评论

编辑 预览

支持 Markdown

自动补全

提交评论 退出

[Ctrl+Enter快捷键提交]

【推荐】100%开源！大型工业跨平台软件C++源码提供，建模，组态！
【推荐】还在用 ECharts 开发大屏？试试这款永久免费的开源 BI 工具！
【推荐】国内首个AI IDE，深度理解中文开发场景，立即下载体验Trae
【推荐】编程新体验，更懂你的AI，立即体验豆包MarsCode编程助手
【推荐】抖音旗下AI助手豆包，你的智能百科全书，全免费不限次数
【推荐】轻量又高性能的 SSH 工具 IShell：AI 加持，快人一步

编辑推荐：
· 一次Java后端服务间歇性响应慢的问题排查记录
· dotnet 源代码生成器分析器入门
· ASP.NET Core 模型验证消息的本地化新姿势
· 对象命名为何需要避免'-er'和'-or'后缀
· SQL Server如何跟踪自动统计信息更新？

阅读排行：
· 官方的 MCP C# SDK：csharp-sdk
· 一款 .NET 开源、功能强大的远程连接管理工具，支持 RDP、VNC、SSH 等多种主流协议！
· 提示词工程师自白：我如何用一个技巧解放自己的生产力
· 一文搞懂MCP协议与Function Call的区别
· 如何不购买域名在云服务器上搭建HTTPS服务

历史上的今天：
2023-03-25 ChatGPT问答[2]-Python类中的方法是闭包吗?是否拥有闭包的性质?
2023-03-25 ChatGPT问答[1]-从信号的合成、分解原理角度，解释傅里叶级数

公告

昵称： qsBye
园龄： 8年5个月
粉丝： 27
关注： 0

< 2025年3月 >

日	一	二	三	四	五	六
23	24	25	26	27	28	1
2	3	4	5	6	7	8
9	10	11	12	13	14	15
16	17	18	19	20	21	22
23	24	25	26	27	28	29
30	31	1	2	3	4	5

搜索

常用链接
我的随笔
我的评论
我的参与
最新评论
我的标签

我的标签
Rust(13)
SLAM(5)
线性代数(2)
LLM(1)

随笔分类
51单片机(10)
Android开发(18)
bouffalo单片机(1)
esp32单片机(24)
GNU Radio(4)
LLM精彩问答(7)
MATLAB(6)
RaspberryPi单片机(6)
SCILAB(3)
stm32单片机(16)
TI单片机(3)
WebAssembly(3)
翻译(5)
高云FPGA(3)
公众号(2)
鸿蒙开发(3)
机器人(11)
前端开发(12)
嵌入式Linux(3)
人工智能(4)
运维相关(16)
折腾笔记(19)

随笔档案
2025年3月(4)
2025年2月(6)
2025年1月(11)
2024年12月(1)
2024年10月(1)
2024年9月(4)
2024年8月(2)
2024年6月(3)
2024年5月(9)
2024年4月(3)
2024年3月(10)
2024年2月(8)
2024年1月(9)
2023年12月(4)
2023年11月(6)
2023年10月(4)
2023年9月(4)
2023年8月(10)
2023年7月(16)
2023年6月(2)
2023年5月(2)
2023年4月(3)
2023年3月(2)
2023年2月(7)
2023年1月(17)
2022年12月(11)
2022年11月(14)
2022年10月(8)
2022年9月(17)
2022年8月(23)
更多

阅读排行榜
1. OpenWrt安装Filebrowser网页文件管理器(11494)
2. esp32笔记[5]-基于I2S协议实现音频播放(9366)
3. 谈谈NAS的硬盘SATA背板(6610)
4. 三星平板的笔记模板(比例:16:10)(4289)
5. 电犀牛R66s软路由体验(4237)

评论排行榜
1. 51单片机笔记[1]-基础实验(1)
2. zemax光学设计笔记[5]-ZPL或ZOS-API的简单使用(1)
3. 小米6安装Ubuntu Touch(1)

推荐排行榜
1. esp32笔记[5]-基于I2S协议实现音频播放(1)
2. esp32笔记[1]-blink闪灯(1)
3. stm32笔记[2]-HAL库驱动2.4寸屏幕(1)
4. 基于esp32的MQTT开关门检测模块(1)
5. 51单片机笔记[6]-电机控制(1)

最新评论
1. Re:zemax光学设计笔记[5]-ZPL或ZOS-API的简单使用
请问博主用vsstudio运行的时候无法读取类型库文件(ZOSAPI.tlb)如何解决啊，ZOSAPII.tlb能用import读取，ZOSAPI.tlb读取却报错，研究了一天了。这两个文件在zosa...
--赤Xin
2. Re:小米6安装Ubuntu Touch
您好！请问应该如何扩容docker呢？我按照本文给出的方案安装之后docker容量只有4G
--VantHoff
3. Re:51单片机笔记[1]-基础实验
大学买的单片机就这样开过.
--mossxzzom

Copyright © 2025 qsBye
Powered by .NET 9.0 on Kubernetes